Avogadro Toast: Módulo 2

Pilar Mujica, Lucia Salamone, Rocio Martinez, Sara Kemelmajer, Caterina Dinnocenzo, and Lucas Segura

Universidad Nacional de Cuyo, Mendoza CP 5502https://uncuyo.edu.ar

Abstract. Este módulo establece los primeros pasos de programación en R-Cran utilizando la plantilla LNCS de Springer.

Keywords: Programming Language · R-Cran · R Markdown

```
knitr::opts_chunk$set(echo = TRUE,
  fig.align = "center",  # Centra todas las figuras
  ↪  automáticamente
  out.width = "100%",    # Fuerza a que no excedan el ancho del
  ↪  margen
  out.extra = "" )
```

1 Primera sección: Aprender a usar R

1.1 Primeros comandos de R

R es un lenguaje muy similiar a mathlab y permite el trabajo con matrices.

```
a=38
a=40
b=60
c=b-a
c
```

```
## [1] 20
```

```
y=40
alpha<-30
```

1.2 Uso de datasets o base de datos intenrnos de R

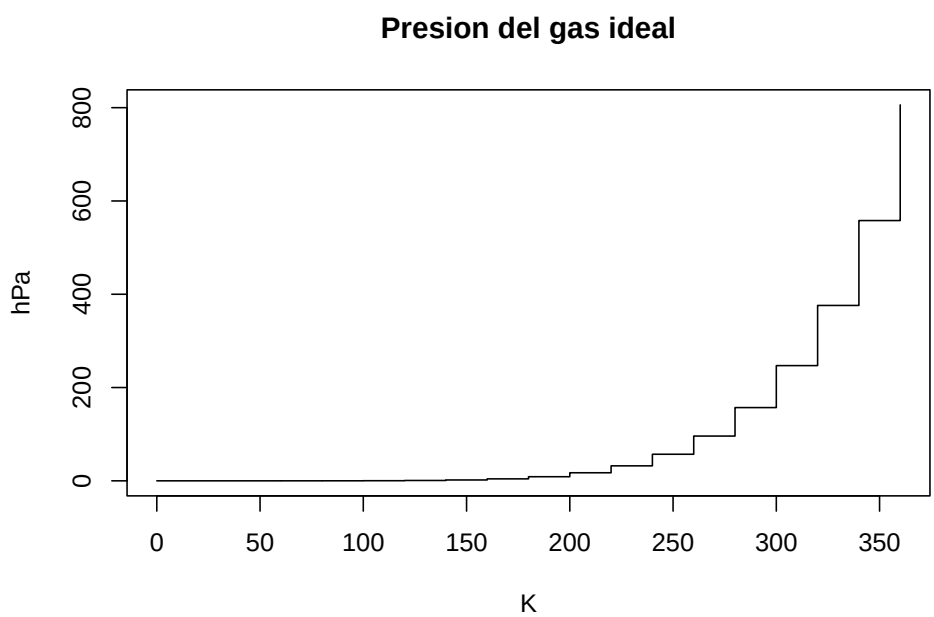
Usando el comando data en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan dados en tablas de las cuales podemos elegir columnas especificas si escribimos $ despues de el comando de informacion especifico y elegimos la columna de datos deseada

```r
summary(cars)
```

```
##      speed           dist       
##  Min.   : 4.0   Min.   :  2.00  
##  1st Qu.:12.0   1st Qu.: 26.00  
##  Median :15.0   Median : 36.00  
##  Mean   :15.4   Mean   : 42.98  
##  3rd Qu.:19.0   3rd Qu.: 56.00  
##  Max.   :25.0   Max.   :120.00  
```

1.3 Comando Plot

Es un comando que permite graficar cualquier fuente de datos que le asignemos, teniendo tambien la posibilidad de asignar nombre a las variables o hacer cambios en el grafico. En caso de duda de como utilizar el comndo podemos escribir ?plot en la consola, lo cual abrira una ventana que explicara en detalle el comando.



1.4 Funciones estadisticas

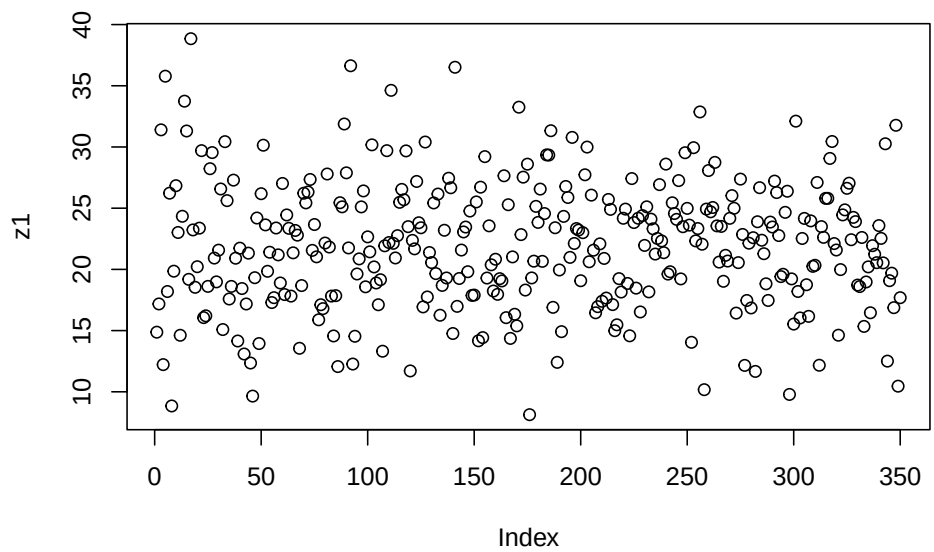
```r
z1<-rnorm(350,22,5)
z1
```

```
##   [1] 14.869709 17.176748 31.398674 12.219307 35.781707 18.191972 26.226232
```

```
##   [8]  8.842567 19.852754 26.837502 23.014415 14.616823 24.338227 33.741390
##  [15] 31.306725 19.185761 38.844539 23.225456 18.535380 20.219015 23.367987
##  [22] 29.700249 16.059933 16.207776 18.610110 28.225355 29.542884 20.925455
##  [29] 18.978983 21.570079 26.568830 15.083871 30.434946 25.617287 17.579929
##  [36] 18.609600 27.291859 20.912664 14.149468 21.734309 18.436251 13.087311
##  [43] 17.173862 21.326057 12.350349  9.648447 19.321107 24.196007 13.941752
##  [50] 26.182258 30.143420 23.634409 19.843211 21.400522 17.291891 17.689455
##  [57] 23.380145 21.186730 18.888939 27.018771 17.944192 24.447431 23.354285
##  [64] 17.817575 21.353012 23.179265 22.796296 13.553477 18.678027 26.221552
##  [71] 25.433474 26.323494 27.344848 21.553197 23.665232 21.025889 15.881205
##  [78] 17.113323 16.794791 22.153511 27.791424 21.826044 17.819292 14.560854
##  [85] 17.853218 12.060554 25.446140 25.111368 31.877280 27.887969 21.753427
##  [92] 36.635553 12.266422 14.535487 19.624533 20.856725 25.105277 26.410022
##  [99] 18.575829 22.657032 21.431538 30.175395 20.200572 18.889073 17.110853
## [106] 19.154950 13.313982 21.926596 29.697938 22.193303 34.625562 22.128102
## [113] 20.933124 22.755643 25.477493 26.534082 25.688005 29.679363 23.492216
## [120] 11.703958 22.369851 21.677557 27.183174 23.810560 23.400357 16.934597
## [127] 30.403788 17.747123 21.377355 20.557193 25.428134 19.657257 26.152886
## [134] 16.253024 18.691144 23.195482 19.298177 27.445965 26.667549 14.760673
## [141] 36.512364 16.979055 19.254721 21.579197 23.057703 23.458180 19.809138
## [148] 24.765810 17.872766 17.900018 25.510157 14.166749 26.717047 14.427257
## [155] 29.211678 19.292485 23.556491 20.373480 18.231434 20.828226 17.961592
## [162] 19.243448 19.056655 27.645839 16.056224 25.283055 14.361399 21.019292
## [169] 16.333194 15.385684 33.250311 22.841185 27.535608 18.311151 28.595543
## [176]  8.131578 19.302222 20.671707 25.147711 23.837058 26.559019 20.655619
## [183] 24.561398 29.354027 29.337868 31.327722 16.899888 23.402326 12.406239
## [190] 19.952202 14.913146 24.330332 26.773736 25.874183 20.980734 30.783302
## [197] 22.116482 23.335505 23.211461 19.073460 22.999634 27.728687 29.992320
## [204] 20.622146 26.067404 21.566825 16.455126 16.954588 22.085043 17.392988
## [211] 20.895942 17.656509 25.711419 24.903764 17.123762 14.991087 15.472448
## [218] 19.245634 18.150204 24.175335 24.924610 18.849718 14.572404 27.419161
## [225] 23.827504 18.468362 24.179927 16.522449 24.394932 21.958457 25.111348
## [232] 18.162264 24.104816 23.315921 21.252580 22.527378 26.924219 22.324687
## [239] 21.364793 28.601255 19.616090 19.807225 25.439998 24.563399 24.086383
## [246] 27.255045 19.222965 23.485903 29.531203 24.985053 23.616232 14.042474
## [253] 29.943428 22.328097 23.337975 32.863016 22.056249 10.173638 24.926270
## [260] 28.076139 24.703798 25.055248 28.740719 23.553111 20.597666 23.509793
## [267] 19.027147 21.149020 20.679928 24.171033 26.024539 24.992670 16.429331
## [274] 20.552313 27.374339 22.853690 12.155994 17.460000 22.141388 16.849612
## [281] 22.589150 11.663523 23.891934 26.675157 22.380820 21.281907 18.822746
## [288] 17.461976 23.863150 23.462338 27.212313 26.274969 22.763719 19.409040
## [295] 19.599704 24.659452 26.383301  9.784899 19.223561 15.531245 32.109426
## [302] 18.195991 16.032550 22.524469 24.163786 18.745837 16.163438 23.962964
## [309] 20.247371 20.334095 27.100109 12.167419 23.492411 22.616175 25.800198
## [316] 25.801510 29.063873 30.446990 22.109407 21.583466 14.622309 19.981871
```

```
## [323] 24.450737 24.862728 26.622279 27.042578 22.423698 24.248789 23.901921
## [330] 18.718148 18.596697 22.606192 15.341725 18.972506 20.206710 16.455779
## [337] 21.938247 21.236114 20.549438 23.577413 22.531706 20.531304 30.261243
## [344] 12.505062 19.078753 19.689392 16.872426 31.772603 10.459199 17.687669
```

```
plot(z1)
```




```
w1<-length(z1)
w1
```

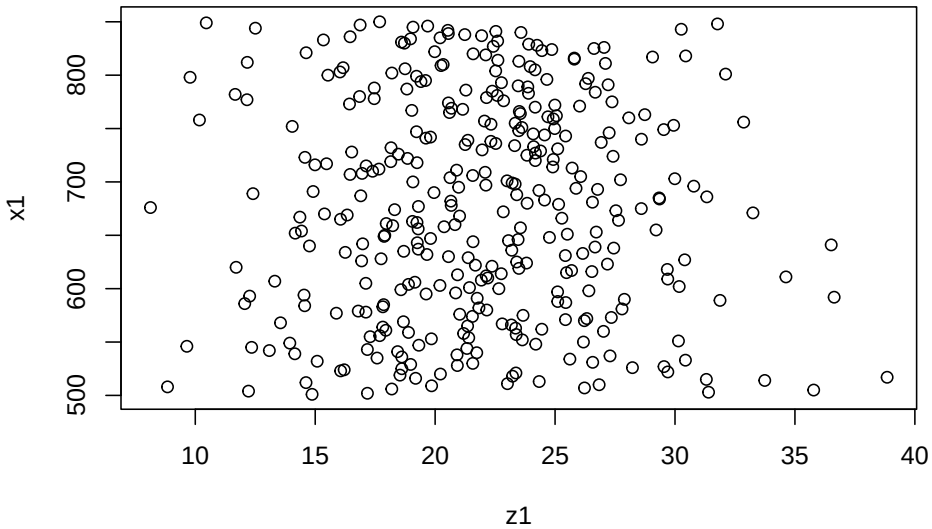
```
## [1] 350
```

```
x1<-(501:850)
x1
```

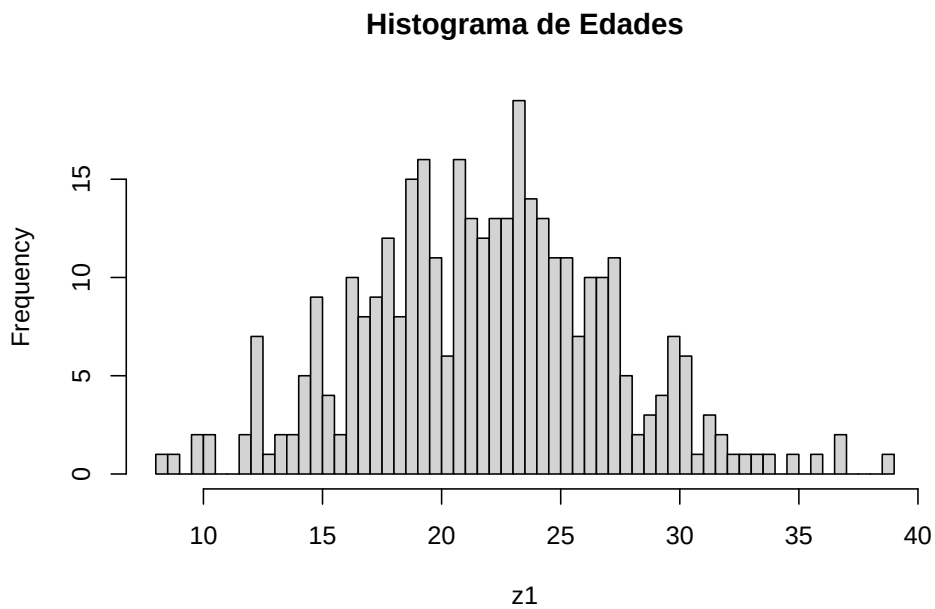
```
##   [1] 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518
##  [19] 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536
##  [37] 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554
##  [55] 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572
##  [73] 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590
##  [91] 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608
## [109] 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626
## [127] 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644
## [145] 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662
## [163] 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680
## [181] 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698
```

```
## [199] 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716
## [217] 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734
## [235] 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752
## [253] 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770
## [271] 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788
## [289] 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806
## [307] 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824
## [325] 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842
## [343] 843 844 845 846 847 848 849 850
```

```r
plot(z1,x1)
```

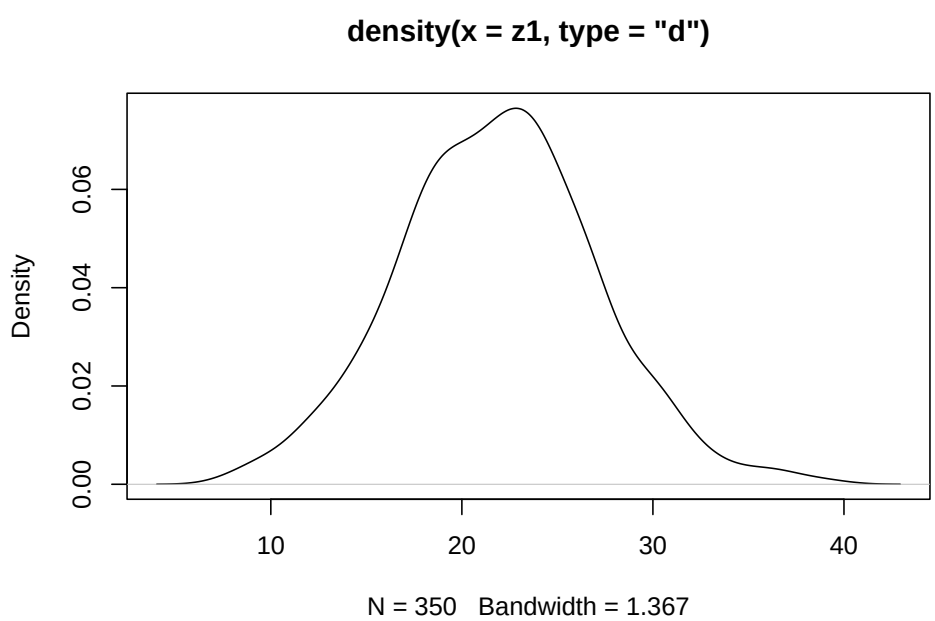



```r
hist(z1,main="Histograma de Edades",breaks=50)
```




```r
plot(density(z1,type="d"))
```

```
## Warning: In density.default(z1, type = "d") :
##  extra argument 'type' will be disregarded
```



2 Segunda sección: Ejercicios en clase:

2.1 Ejercicio 1 de Algoritmos

consigna: las ultimas 3 cifras del DNI son 195, consigna: crear una variable que tenga sus numeros

```
DNI<-195
```

Consigna: Crear un vector que tenga todos los numeros enteros desde el 1 al 195.

```
Secuencia_dni<-(1:195)
Secuencia_dni
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195
```

2.2 Ejercicio 2: Calcular la suma de todos los valores del vector secuencia_dni

```
Total<-0
Valor_Final<-length(Secuencia_dni)
for(i in 1:Valor_Final) {
  Total<-Total+i
  Total}
```

2.3 Ejercicio 3

Repetir el ejercicio anterior pero en Python

```
secuencia_dni = range(1, 196)
suma = sum(secuencia_dni)

print(f"La suma total es: {suma}")
```

```
## La suma total es: 19110
```

2.4 Ejercicio 4

¿Cuanto tarda en correr el codigo del ejercicio 2?

```
Inicio<-Sys.time()
Total<-0
Valor_Final<-1000000*length(Secuencia_dni)
for(i in 1:Valor_Final) {
  Total<-Total+i
  Total}

Final<-Sys.time()
Final-Inicio
```

```
## Time difference of 3.707734 secs
```

2.5 Ejercicio 5

Repetir ejercicio anterior utilizando la cabeza de Gauss

```
n <- 195
suma_gauss <- (n * (n + 1)) / 2

print("La suma usando la fórmula de Gauss es:")
```

```
## [1] "La suma usando la fórmula de Gauss es:"
```

```
suma_gauss
```

```
## [1] 19110
```

3 Tercera sección: Ejercicios del PDF:

3.1 1.4 Generar un vector secuencia

```
tiempo_for <- system.time({
  A <- numeric(50000) # Pre-asignar espacio es una buena práctica
↪  en R
  for (i in 1:50000) {
    A[i] <- (i * 2)
  }
})
# Mostrar extremos del vector resultante
print("Extremo Inicial (head):")
```

Método 1: Uso de bucle 'for' (anidamiento clásico)

```
## [1] "Extremo Inicial (head):"
```

```r
print(head(A))
```

```
## [1]  2  4  6  8 10 12
```

```r
print("Extremo Final (tail):")
```

```
## [1] "Extremo Final (tail):"
```

```r
print(tail(A))
```

```
## [1]  99990  99992  99994  99996  99998 100000
```

```r
tiempo_seq <- system.time({
  B <- seq(from = 2, to = 100000, by = 2)
})
# Mostrar extremos del vector resultante
print("Extremo Inicial (head):")
```

Método 2: Uso del comando nativo de R 'seq'

```
## [1] "Extremo Inicial (head):"
```

```r
print(head(B))
```

```
## [1]  2  4  6  8 10 12
```

```r
print("Extremo Final (tail):")
```

```
## [1] "Extremo Final (tail):"
```

```r
print(tail(B))
```

```
## [1]  99990  99992  99994  99996  99998 100000
```

```r
print("Performance del bucle FOR:")
```

Comparativa de Performance

```
## [1] "Performance del bucle FOR:"
```

```r
print(tiempo_for)
```

```
##    user  system elapsed
##   0.005   0.000   0.004
```

```r
print("Performance del comando SEQ:")
```

```
## [1] "Performance del comando SEQ:"
```

```r
print(tiempo_seq)
```

```
##    user  system elapsed
##   0.001   0.000   0.000
```

3.2 Ejercicio 1.5 e 1.6 Implementación de Fibonacci

```r
# Inicializamos el vector con los dos primeros términos
fib <- c(0, 1)
limite <- 1000000
iteracion <- 2 # Ya tenemos los dos primeros elementos

# Bucle para generar la serie hasta superar el millón
while (fib[length(fib)] <= limite) {
# El nuevo término es la suma de los dos anteriores
  nuevo_termino <- fib[length(fib)] + fib[length(fib) - 1]

  fib <- c(fib, nuevo_termino)

  iteracion <- iteracion + 1

  cat("Iteración:", iteracion, " - Valor generado:",
  ↪  nuevo_termino, "\n")
}
```

```
## Iteración: 3  - Valor generado: 1
## Iteración: 4  - Valor generado: 2
## Iteración: 5  - Valor generado: 3
## Iteración: 6  - Valor generado: 5
## Iteración: 7  - Valor generado: 8
## Iteración: 8  - Valor generado: 13
## Iteración: 9  - Valor generado: 21
## Iteración: 10  - Valor generado: 34
## Iteración: 11  - Valor generado: 55
## Iteración: 12  - Valor generado: 89
## Iteración: 13  - Valor generado: 144
## Iteración: 14  - Valor generado: 233
## Iteración: 15  - Valor generado: 377
## Iteración: 16  - Valor generado: 610
## Iteración: 17  - Valor generado: 987
## Iteración: 18  - Valor generado: 1597
## Iteración: 19  - Valor generado: 2584
## Iteración: 20  - Valor generado: 4181
## Iteración: 21  - Valor generado: 6765
## Iteración: 22  - Valor generado: 10946
## Iteración: 23  - Valor generado: 17711
## Iteración: 24  - Valor generado: 28657
## Iteración: 25  - Valor generado: 46368
## Iteración: 26  - Valor generado: 75025
## Iteración: 27  - Valor generado: 121393
## Iteración: 28  - Valor generado: 196418
## Iteración: 29  - Valor generado: 317811
## Iteración: 30  - Valor generado: 514229
## Iteración: 31  - Valor generado: 832040
## Iteración: 32  - Valor generado: 1346269
```

```r
cat("RESULTADO FINAL:\n")
```

```
## RESULTADO FINAL:
```

```r
cat("Se necesitaron", iteracion, "iteraciones.\n")
```

```
## Se necesitaron 32 iteraciones.
```

```r
cat("El primer número mayor a", limite, "es:", fib[length(fib)],
↪  "\n")
```

```
## El primer número mayor a 1e+06 es: 1346269
```

```r
cat("Serie completa generada:\n")
```

```
## Serie completa generada:
```

```r
print(fib)
```

```
##  [1]       0       1       1       2       3       5       8      13      21
## [10]      34      55      89     144     233     377     610     987    1597
## [19]    2584    4181    6765   10946   17711   28657   46368   75025  121393
## [28]  196418  317811  514229  832040 1346269
```

3.3 Ejercicio 1.7 Ordenacion de un vector por método burbuja

```r
library(microbenchmark)

burbuja <- function(x){
  n <- length(x)
  for(j in 1:(n-1)) {
    for(i in 1:(n-j)){
      if(x[i] > x[i+1]){
        temp <- x[i]
        x[i] <- x[i+1]
        x[i+1] <- temp
      }
    }
  }
  return(x)
}

# Generar la muestra de tamaño 20,000
set.seed(2026)
muestra_grande <- sample(1:100000, 20000)

# Medir performance
res_comparativo <- microbenchmark(
  "Burbuja (Manual)" = burbuja(muestra_grande),
  "Sort (Nativo R)" = sort(muestra_grande),
  times = 1
)
# Mostrar resultados
cat("\n Resultados con n = 20,000 \n")
```

```
##
##  Resultados con n = 20,000
```

```r
print(res_comparativo)
```

```
## Unit: microseconds
##              expr          min           lq         mean       median
##  Burbuja (Manual) 21099765.778 21099765.778 21099765.778 21099765.778
##   Sort (Nativo R)      863.941      863.941      863.941      863.941
##            uq          max neval
##  21099765.778 21099765.778     1
##       863.941      863.941     1
```

3.4 Ejercicio 1.8: La penitencia de Newton

```r
library(tictoc)

# Definimos el límite de la suma (diez mil millones)
n_objetivo <- 1e10

cat("Iniciando Comparativa para n =", n_objetivo, "\n\n")
```

```
## Iniciando Comparativa para n = 1e+10
```

```r
# MÉTODO 1: Algoritmo de Sumas Parciales (Bucle For)


tic("Tiempo del Bucle")
suma_bucle <- 0
for(i in 1:n_objetivo) {
  suma_bucle <- suma_bucle + i
}
toc()
```

```
## Tiempo del Bucle: 179.257 sec elapsed
```

```r
# MÉTODO 2: Algoritmo de la Función Matemática (Gauss)

cat("\n2. Ejecutando Fórmula Matemática (con el n real =
↪  1e10)...\n")
```

```
##
## 2. Ejecutando Fórmula Matemática (con el n real = 1e10)...
```

```r
tic("Tiempo de la Fórmula")
# Usamos la fórmula: (n * (n + 1)) / 2
suma_formula <- (n_objetivo * (n_objetivo + 1)) / 2
toc()
```

```
## Tiempo de la Fórmula: 0.001 sec elapsed
```

```r
# RESULTADOS FINALES

cat("Resultado final de la suma de Newton:", suma_formula, "\n")
```

```
## Resultado final de la suma de Newton: 5e+19
```

```r
cat("Conclusión: El benchmarking nos permite entender la calidad
del algoritmo y la eficiencia según el recurso físico.\n")
```

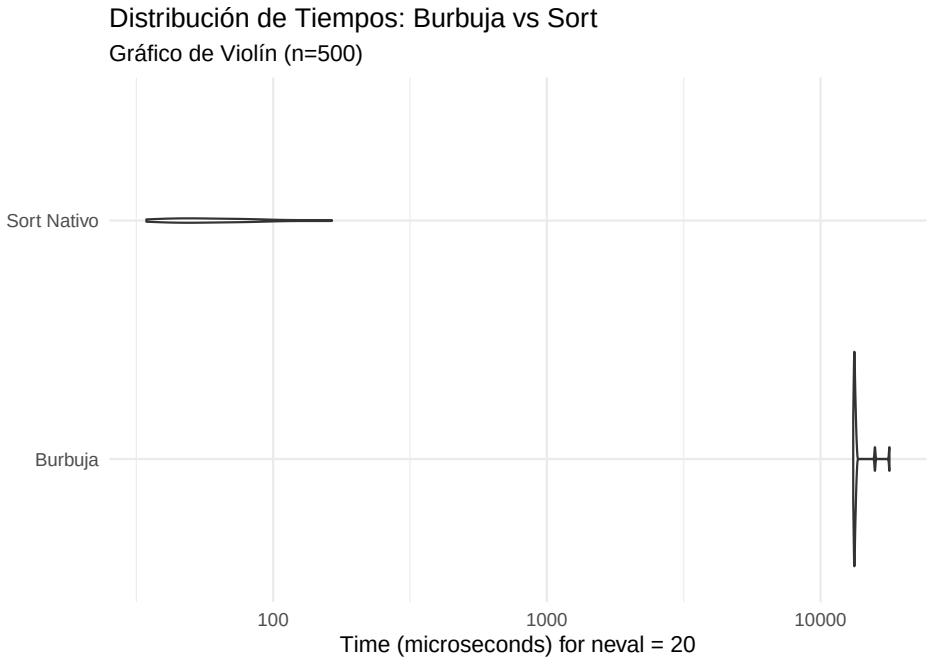
```
## Conclusión: El benchmarking nos permite entender la calidad
## del algoritmo y la eficiencia según el recurso físico.
```

4 Cuarta sección:Benchmarking de Algoritmos y Medioides (kmeans)

4.1 Comparativa métodos de ordenamiento "burbuja" y sort nativo de R .

```r
library(microbenchmark)
library(ggplot2)

# 1. Definimos la función de ordenamiento burbuja
burbuja <- function(x){
  n <- length(x)
  for(j in 1:(n-1)) {
    for(i in 1:(n-j)){
      if(x[i] > x[i+1]){
        temp <- x[i]; x[i] <- x[i+1]; x[i+1] <- temp
      }
    }
  }
  return(x)
}

# 2. Generamos una muestra de 500 números (más ágil)
```

```r
set.seed(123)
datos_v <- sample(1:10000, 500)

# 3. Medimos la performance (20 repeticiones)
bm_ordenacion <- microbenchmark(
  "Burbuja" = burbuja(datos_v),
  "Sort Nativo" = sort(datos_v),
  times = 20
)

# 4. Creamos el gráfico y usamos PRINT para forzar su aparición
p1 <- autoplot(bm_ordenacion) +
  labs(title = "Distribución de Tiempos: Burbuja vs Sort",
       subtitle = "Gráfico de Violín (n=500)",
       x = "Algoritmo", y = "Tiempo") +
  theme_minimal()

print(p1)
```



Conclusión: Comparativa de Ordenamiento Como se observa en el gráfico de violín anterior, el método **Sort Nativo** de R es órdenes de magnitud más eficiente que el algoritmo de **Burbuja**. Mientras que el Sort nativo opera en microsegundos con una variabilidad casi nula, el bucle manual de Burbuja re-

quiere un tiempo significativamente mayor debido a su complejidad algorítmica $O(n^2)$ y al procesamiento línea por línea en R. Esto demuestra la importancia de utilizar funciones vectorizadas y nativas para el manejo de grandes volúmenes de datos.

4.2 Análisis de Performance K-means

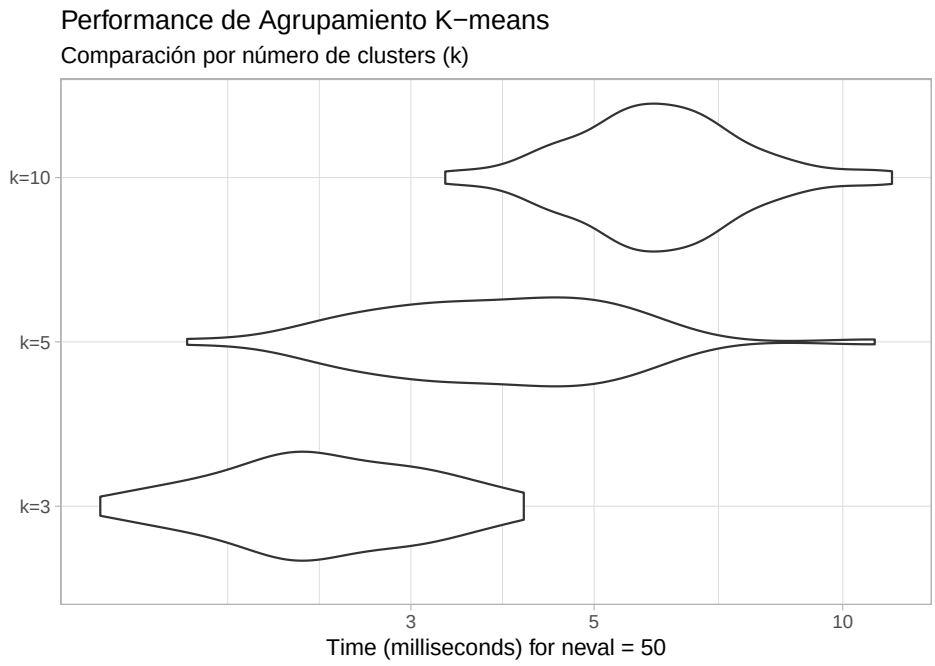
Introducción Teórica El algoritmo **K-means** es un método de aprendizaje no supervisado utilizado para el agrupamiento (*clustering*). Su objetivo es particionar un conjunto de n observaciones en k grupos distintos. El algoritmo funciona de manera iterativa: 1. Se inicializan k centroides de forma aleatoria. 2. Cada punto se asigna al centroide más cercano (basado en la distancia euclídea). 3. Se recalculan los centroides promediando la posición de todos los puntos asignados a cada grupo. 4. El proceso se repite hasta que las asignaciones no cambian o se alcanza un límite de iteraciones.

La eficiencia del método es alta, pero su tiempo de ejecución se ve afectado por el número de dimensiones de los datos y, fundamentalmente, por la cantidad de clusters (k) definidos.

4.3 Medición de Performance con Gráfico de Violín

```r
library(microbenchmark)
library(ggplot2)

# 1. Simulamos un dataset de 5.000 puntos en 2 dimensiones
set.seed(42)
datos_km <- matrix(rnorm(10000), ncol = 2)

# 2. Medimos el tiempo para encontrar 3, 5 y 10 clusters
# Realizamos 50 repeticiones para obtener una distribución
↪  robusta
bm_kmeans <- microbenchmark(
  "k=3" = kmeans(datos_km, centers = 3),
  "k=5" = kmeans(datos_km, centers = 5),
  "k=10" = kmeans(datos_km, centers = 10),
  times = 50
)

# 3. Generación del gráfico de violín
p_km <- autoplot(bm_kmeans) +
  labs(title = "Performance de Agrupamiento K-means",
       subtitle = "Comparación por número de clusters (k)",
       x = "Cantidad de Grupos (k)", y = "Tiempo de ejecución") +
  theme_light()
```

```r
# 4. Forzar visualización en el PDF
print(p_km)
```



4.4 Conclusión K-means

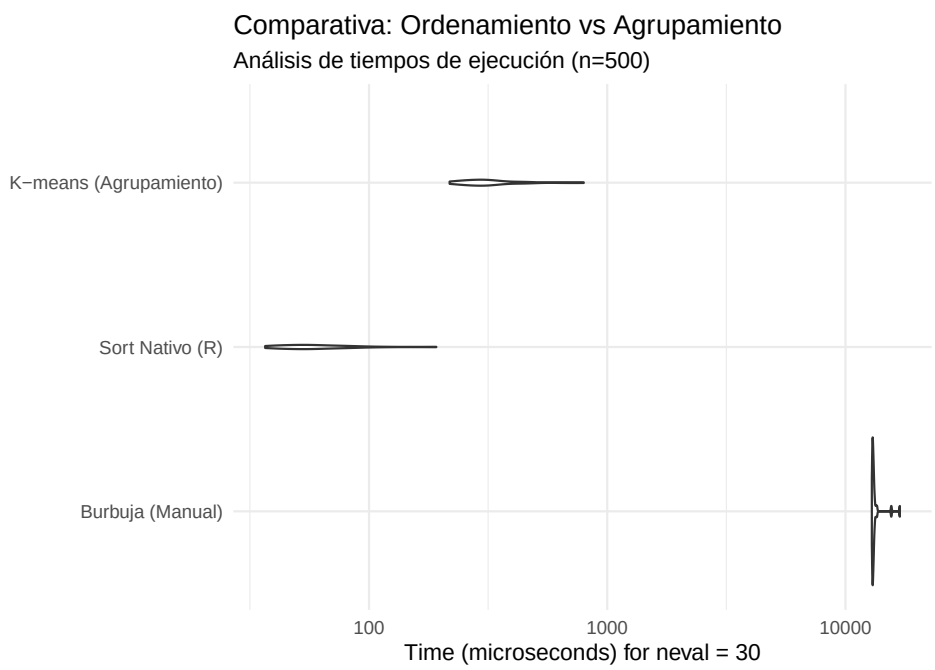
El gráfico resultante permite observar cómo el tiempo de ejecución se incrementa a medida que aumentamos el valor de k. Esto se debe a que el algoritmo debe realizar una mayor cantidad de cálculos de distancia entre cada punto y los centroides en cada una de las iteraciones del proceso.

4.5 Comparativa Global de Algoritmos

En esta sección final, comparamos el rendimiento de tres procesos distintos: el ordenamiento manual (**Burbuja**), el ordenamiento optimizado (**Sort Nativo**) y un proceso de inteligencia artificial no supervisada (**K-means**). El objetivo es observar cómo la eficiencia del código impacta más que la complejidad de la tarea en sí.

```r
library(microbenchmark)
library(ggplot2)

# Preparación de datos
```

```r
set.seed(123)
datos_vec <- sample(1:5000, 500)
datos_mat <- matrix(rnorm(1000), ncol = 2)

# Ejecución del benchmark
comparativa_total <- microbenchmark(
  "Burbuja (Manual)" = burbuja(datos_vec),
  "Sort Nativo (R)" = sort(datos_vec),
  "K-means (Agrupamiento)" = kmeans(datos_mat, centers = 3),
  times = 30
)

# Renderizado del gráfico
p_final <- autoplot(comparativa_total) +
  labs(title = "Comparativa: Ordenamiento vs Agrupamiento",
       subtitle = "Análisis de tiempos de ejecución (n=500)",
       x = "Algoritmo", y = "Tiempo (Escala Logarítmica)") +
  theme_minimal()

print(p_final)
```



4.6 Conclusión Final

A partir de los experimentos realizados en este módulo, se desprenden tres conclusiones fundamentales:

1. **La eficiencia del lenguaje nativo:** El método `sort` de R es el más rápido por amplio margen. Esto confirma que las funciones internas de R, escritas en lenguajes de bajo nivel (C/Fortran), están altamente optimizadas para el procesamiento de datos.
2. **El costo de la ineficiencia:** El algoritmo de **Burbuja**, a pesar de realizar una tarea más simple que el agrupamiento, resulta ser el más lento de los tres. Esto demuestra que un algoritmo mal diseñado ($O(n^2)$) puede ser menos eficiente que un proceso de IA complejo pero bien implementado.
3. **Complejidad vs. Rendimiento:** El método **K-means** se sitúa en un punto intermedio. Aunque realiza cálculos matemáticos avanzados para encontrar centroides, su implementación nativa en R permite que procese grandes volúmenes de datos en un tiempo razonable, superando significativamente a los bucles `for` manuales.

En definitiva, para el desarrollo de herramientas modernas, es imperativo priorizar funciones vectorizadas y algoritmos optimizados sobre implementaciones manuales iterativas.